

HomeWork #1

Question 1:

1- Explain the difference between and provide real life example:

- Class
- Instance

2- Why do we use **getter and setter methods** instead of making attributes public? Support your answer with a practical scenario.

3- When should we use:

- A constructor without parameters
- A constructor with parameters

Give one example for each.

4- Explain the difference between:

- A regular method
- A static method

When can using a static method be considered bad design?

5- What **toString()** method return and How can I use for change the return data, and give me code example

Question 2

1- Analyze the following class:

```
class Student {  
    String name;  
    int grade;  
}
```

- What are the design problems in this class?
- Improve it using proper OOP principles.

2-Consider the following code:

```
3 public class JavaTest {  
5  
6     Run main | Debug main | Run | Debug  
7     public static void main(String[] args) {  
8  
9         Student s1 = new Student(name: "Ali");  
10        Student s2 = s1;  
11        s2.setName(name: "Dalia");  
12    }  
13 }
```

- What will be the value of s1.name?
- Explain your answer using the concept of references.

3-Analyze the equals method:

```
public boolean equals(Student s) {  
    return this == s;  
}
```

- -Is this implementation correct?
- If not, rewrite it correctly and explain why.

Question 3 :

Explain the difference between:

- Reference copy
- Actual (deep) copy

Question 4

Read the following description carefully, then complete the required tasks.

A **Library** system contains multiple **Book** objects and multiple **Member** objects. Each Library has a name and a location.

A **Book** has a title, an author, and a price.

A **Member** has a name and a member ID, and can borrow or return books.

The relationship between **Library and Book**, and between **Library and Member**, is an **Aggregation relationship**, meaning that a Library **has-a** collection of Books and Members, but they can exist independently.

-Requirements:

* Draw a UML diagram based on the description

• Include:

- All classes
- Attributes and methods
- Clearly show **Aggregation relationships (has-a)**

*Java Implementation

*******تعليمات تسليم الواجب*******

1. يجب حل الواجب **بخط اليد فقط** وتسليمه كملف **PDF واحد واضح**
2. **آخر موعد للتسليم: 5/15** ولن يُقبل أي تسليم بعد هذا التاريخ .
3. يجب أن تكون الإجابات **بأسلوب الطالب الخاص** مع شرح واضح، ويُمنع النسخ أو الاعتماد على إجابات جاهزة .
4. **سيتم اختيار 10 طلاب عشوائيًا لعمل مناقشة شفوية حول حل الواجب وشرح بعض الإجابات .**
5. التقييم يعتمد على **الفهم، وضوح الحل، والتنظيم** وليس الحفظ فقط .
6. يجب تسمية الملف بالشكل التالي: **اسم الطالب_الرقم_OOP.pdf_**

